

School of Computer Science and Engineering

WHERE IDEAS IGNITE MINDS

Laboratory Manual Embedded Systems & Microcontrollers

Bachelor of Technology (B.Tech.) (Hons.)

Academic Year 2025-2026



RV UNIVERSITY
SCHOOL OF COMPUTERS SCIENCE AND ENGINEERING
BENGALURU- 560059



LABORATORY CERTIFICATE

This is to certify that

Mr./Ms. _____ has

satisfactorily completed the course of experiments in Practical Embedded Systems and Microcontrollers (CS1102) prescribed by the School of Computer Science and Engineering during the year 2025-26.

USN: _____ Semester: _____

Marks	
Maximum	Obtained
15	

Faculty In-charge

Date

Program Director

Vision and Mission of the School of Computer Science and Engineering

Vision

To be a pioneering school of Computer Science and Engineering committed to fostering liberal education and empowering the next generation of technologists to make a positive global socio-economic impact.

Mission

- To be a pioneer in computer science education and benchmark ourselves with the world's top computer science and engineering institutions.
- To provide state-of-the-art facilities that enable exemplary pedagogy, advanced research, innovation and entrepreneurship in emerging technologies of computer science.
- To promote a culture of cooperation and inclusiveness among students and faculty from diverse communities enabling them to take part in interdisciplinary and multidisciplinary research, contributing to institution-building.
- To foster excellence through national and international academic, industry collaborations, bringing in diverse perspectives to drive innovation.
- To nurture a talented pool of ethical, self-driven and empathetic problem solvers to achieve sustainable development goals.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will be able to demonstrate excellence in the field of Computer Science and Engineering through advanced research and entrepreneurship.

PEO2: Graduates will be able to achieve excellence in innovation through national and international academic and industry collaborations.

PEO3: Graduates will be able to actively engage in cutting-edge interdisciplinary and multidisciplinary research.

PEO4: Graduates will be able to function effectively as a talented pool of ethical, sustainable, self-driven and life-long empathetic problem solvers.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

PO1: Engineering Knowledge. Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: *Problem Analysis. Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.*

PO3: *Design/Development of Solutions. Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.*

PO4: *Conduct Investigations of Complex Problems. Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.*

PO5: *Modern Tool Usage. Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.*

PO6: *The Engineer and Society. Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues, and the consequent responsibilities relevant to the professional engineering practice.*

PO7: *Environment and Sustainability. Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.*

PO8: *Individual and Teamwork. Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.*

PO9: *Ethics. Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.*

PO10: *Communication. Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.*

PO11: *Project Management and Finance. Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, managing projects and in multidisciplinary environments.*

PO12: *Life-Long Learning. Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.*

Course Outcomes: After completing the course, the students will be able to:	
CO1	Understand the concept of embedded system, microcontroller, different components of microcontroller and their interactions as applied to RP2040.
CO2	Get familiarized with programming environment to develop embedded solutions
CO3	Program ARM microcontroller to perform various tasks and stack implementation in Cortex-M0+.
CO4	Understand the key concepts of embedded systems such as Instruction cycle execution, I/O, DMA, interrupts and interaction with peripheral devices.

Course Outcome (CO) – Program Outcome (PO) Mapping												
C Os	PO1	PO 2	PO3	PO4	PO 5	PO6	PO7	P O 8	PO9	PO10	PO11	PO 12
	Engi neer ing kno wled ge	Pro ble m ana lysi s	Develop ment of solution s	Conduct investig ations	Mo der n too l usa ge	Engi neer and soci ety	Environ ment and sustaina bility	E th ic s	Indiv idual and team work	Communi cations	Project manage ment	Life lon g lear nin g
C O1	3	2	-	-	1	-	-	-	2	-	-	3
C O2	3	2	2	1	3	-	-	-	1	-	-	3
C O3	3	2	3	1	3	-	-	-	2	2	-	3
C O4	3	2	-	1	2	-	-	-	-	-	-	3

Justification for CO-PO Mapping:

CO1: Understand the concept of embedded system, microcontroller, different components of microcontroller and their interactions as applied to RP2040.

- PO1 (Engineering knowledge): Understanding basic embedded systems and microcontroller concepts requires strong foundational knowledge in engineering. (Level 3)
- PO2 (Problem analysis): Analyzing how microcontroller components interact involves problem-solving skills. (Level 2)
- PO5 (Modern tool usage): Understanding embedded systems involves using modern tools and technologies. (Level 1)
- PO12 (Lifelong learning): Continuous learning is needed to keep up with advancements in embedded systems. (Level 1)

CO2: Get familiarized with programming environment to develop embedded solutions.

- PO1 (Engineering knowledge): Familiarization with programming environments builds on engineering fundamentals. (Level 3)
- PO2 (Problem analysis): Debugging and troubleshooting embedded programs require analytical skills. (Level 2)
- PO3 (Development of solutions): Developing embedded solutions directly contributes to creating solutions. (Level 2)
- PO4 (Conduct investigations): Investigating programming issues and solutions enhances investigative skills. (Level 1)
- PO5 (Modern tool usage): Using modern programming environments and tools. (Level 3)
- PO9 (Individual and team work): Collaborating in programming projects develops teamwork skills. (Level 1)
- PO10 (Communications): Communicating programming issues and solutions effectively. (Level 1)
- PO11 (Project management): Managing programming tasks within projects. (Level 1)
- PO12 (Lifelong learning): Keeping up with new programming tools and environments requires lifelong learning. (Level 2)

CO3: Program ARM microcontroller to perform various tasks and stack implementation in Cortex-M0+.

- PO1 (Engineering knowledge): Programming ARM microcontrollers requires applying engineering knowledge. (Level 3)
- PO2 (Problem analysis): Analyzing programming issues in ARM microcontrollers. (Level 2)
- PO3 (Development of solutions): Developing programs for ARM microcontrollers contributes to creating solutions. (Level 3)
- PO4 (Conduct investigations): Investigating ARM microcontroller programming issues. (Level 1)
- PO5 (Modern tool usage): Using tools specific to ARM programming. (Level 3)
- PO12 (Lifelong learning): Continuously learning new programming techniques and updates in ARM architecture. (Level 2)

CO4: Understand the key concepts of embedded systems such as Instruction cycle execution, I/O, timers interrupts and interaction with peripheral devices.

- PO1 (Engineering knowledge): Understanding key concepts of embedded systems builds on foundational engineering knowledge. (Level 3)
- PO2 (Problem analysis): Analyzing instruction cycles and I/O operations requires problem-solving skills. (Level 2)
- PO4 (Conduct investigations): Investigating embedded system operations. (Level 1)
- PO5 (Modern tool usage): Using tools to understand and interact with embedded systems. (Level 2)
- PO12 (Lifelong learning): Keeping up with new developments in embedded systems requires lifelong learning. (Level 1)

INDEX

Sl. No.		Lab Activities
1	1.1	Blink the LED_BUILTIN using a blink LED program from examples.
	1.2	Display “Hello World “Using the Serial monitor tool.
	1.3	Write a program to blink 2 alternate LEDs out of 4 LEDs with delay of 3 and 5 seconds respectively
2	2.1	Write a program to display Numbers in the increasing and decreasing order from 1 to 15 and vice versa in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value (15), start counting down to 1, then again begin counting up to 15, then start from 1. Whenever it comes to the Max or Min value, wait for 2 seconds.
	2.2	Write a program to display odd, even numbers in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value, wait 2 seconds and start from the minimum. (Example: 0001, 0010, 0011, ..., 1111, 1110, 1101, ..., 0001,0010,0011, ...)
3	3.1	Write an assembly function that just returns 200 back to the C program and write an assembly function that returns sum of three constants 10, 20 and 30, and returns the result.
	3.2	Write an assembly function that returns sum of four constants 10, 20, 30 and 40, and returns the result.
	3.3	Write an assembly function that returns sum of five constants 10, 20, 30, 40 and 50, and returns the result.
4	4.1	Write an ARM assembly function that takes three constants as inputs from C program, perform ADD operation, and returns the result to the C program.
	4.2	Write an ARM assembly function that takes two input parameters, performs an AND, OR logic operation on it, and returns the result to a calling C program.
	4.3	Write an ARM assembly function that takes one input parameter, performs a bitwise NOT operation on it, and returns the result to a calling C program.
5	5.1	Write an ARM assembly function that performs 2 times logical shift right operation on one parameter and returns the result to a calling C program.
	5.2	Write an ARM assembly function that performs 2 times logical shift right and 4 times logical shift left operation on one parameter and returns the result to a calling C program
6		Write an ARM assembly function that performs 8 times arithmetic shift right operation on one signed (Negative number) parameter and returns the result to a calling C program.
7	7.1	Verify the De-morgans theorem with two inputs. If the result is TRUE, print both the results and “De-morgans theorem is verified”, else, print both the results and “De-morgans theorem is not verified”.
	7.2	Perform EX-OR operation with two inputs. If the result is TRUE, print both the results and “EX-OR logic expression is verified”, else, print both the results and “EX-OR logic expression is not verified”.
8	8.1	Write ARM assembly functions to take two unsigned parameters and perform an ADD operation.
	8.2	Write ARM assembly functions to Read the flag bits from the PSR. Return both outputs to the main function.

9	9	Write a program for the Raspberry Pi Pico to transmit and receive data through serial communication (UART) port. Transmitting and Receiving Board (RP2040): UART0, GP0 (UART0 Tx), GP1 (UART0 Rx)
10	10	Write a program to control one slave device from one master device using serial communication through the I2C port with the following specifications. Master Slave Board (RP2040): I2C0, GPIO pins: GP0 (I2C0 SDA), GP1 (I2C0 SCL)

Introduction Embedded systems are specialized computing systems that perform dedicated functions within larger systems. Unlike general-purpose computers, embedded systems are designed for specific control functions often with real-time computing constraints. Microcontrollers are integral components of these systems, providing the processing power and interfaces required to interact with other components and peripherals.

Guidelines

1. The class will be divided into groups of 3 students who will be working together during laboratory hours.
2. Every student must submit their laboratory records on time.
3. You are expected to perform the experiment, complete the calculations and get it verified by the instructor on the same day. The report of the same MUST be submitted before performing the next experiment.
4. At least one set of observations should be signed by the instructor. An unsigned data (observation) will not be graded.
5. You must bring with you the following materials: Data Sheet, Laboratory Manual, Pen, Pencil, Graph Sheets, Calculator and other stationary items as required.
6. ALL DATA SHOULD BE RECORDED WITH PEN ONLY. Pencils are to be used only for graphs and schematic diagrams.
7. You are expected to give a careful thought on the precautions to be observed in handling any equipment and conducting the experiment. In case of any doubt, you should feel free to interact with the instructors.
8. If you are absent in any lab class, you must produce valid reason in the next class to be eligible to perform the missed-out experiment in makeup class.
9. Late submission of the records will not be tolerated and will lead to loss of marks.

Laboratory test

Before the test

1. You are required to have performed all the experiments listed in the course.
2. Your Laboratory record should be signed by the lab in-charge of all the performed experiments.

On the day of test

Requirements

1. You are required to bring with you the signed laboratory record. [You will NOT be allowed to take the test without a signed record].
2. You are required to bring with your necessary stationary items including but not restricted to Pen (Blue or Black), Pencils, Calculator, Ruler, Eraser, Sharpener etc.

Conduct:

On the day of the test, you will be performing one experiment that will be allotted to you by selection of chits. You are then required to:

1. Submit your laboratory record to the lab instructor.
2. Head to your station as instructed by faculty and lab instructor.
3. You are required to demonstrate the basic understanding of the experiment you will be performing by providing the outline of the experiment on the answer sheet provided.
4. Get the outline verified by internal/external examiner.
5. You can then proceed to perform the experiment given to you.
6. You are required to call upon the examiners to verify any ONE observation obtained.
7. After verification of the observation, you are to complete the experiment and submit the answer sheet.
8. Viva will be conducted after submission of the answer sheet.

After the test

After completion of the test, you are required to tidy your station and hand over the lab components to the instructor.

Rubrics for Lab Write-up and Execution (Max: 5+5 = 10 marks)						
LAB WRITE-UP (5 Marks)						
Sl. No.	Criteria	Measuring Method	Excellent	Good	Satisfactory	Poor
1	Lab Record write up (1 Marks)	Submission of lab record	Complete write up of Lab programs with proper syntax. (1 Marks)	Complete write up of Lab programs with minor issues (0.75 Mark)	Complete write up of Lab programs with some syntax errors (0.5 Marks)	No submission of Lab Record (0 Marks)
2	Interfacing Hardware board (1 Marks)	Observation	The hardware circuit is assembled accurately, neatly, and operates efficiently as intended. (1 Marks)	The circuit functions correctly with minor wiring or implementation issues.. Results are interpreted correctly. (0.75 Mark)	The circuit is partially functional and shows noticeable errors or inefficiencies. (0.5 Marks)	The circuit is incorrect, unsafe, or non-functional. Results are misinterpreted. (0 Mark)
3	Software Development (2 Mark)	Code Review	The program code is efficient, well-structured, fully functional, and clearly commented. (2 Marks)	The code functions correctly with minor inefficiencies or limited documentation. (1.5Mark)	The code is functional but lacks efficiency, clarity, or sufficient commenting. (1 Mark)	The code is inefficient, poorly documented, or non-functional. (0 Mark)
4	Interpretation of Result and Insight (1 Mark)	Observation	Results are interpreted insightfully, clearly linked to the experiment objectives, and demonstrate strong conceptual understanding. (1 Marks)	Interpretation is clear and mostly aligned with the objectives. (0.75 Marks)	Interpretation is adequate but shows limited connection to the objectives. (0.5 Marks)	Interpretation is unclear and poorly related to the objectives. (0 Mark)

VIVA VOCE (5 Marks)						
1	Understanding of Concepts (3 Marks)	Viva	Demonstrates a thorough understanding of core concepts and explains them clearly and confidently. (3 Marks)	Shows good understanding of core concepts with minor misconceptions. (2 Marks)	Demonstrates partial understanding of core concepts with several misconceptions. (1 Mark)	Lacks understanding of core concepts and is unable to explain them clearly. (0 Mark)
2	Response to Questions (2 Marks)	Viva	Responds accurately, confidently, and thoughtfully, providing detailed and well-reasoned answers.. (2 Marks)	Responds accurately and confidently, with minor details missing. (1.5 Mark)	Responds with some inaccuracies or hesitation; answers are often incomplete or lack sufficient detail. (1 mark)	Struggles to respond accurately or confidently; answers are largely incorrect, incomplete, or unclear. (0Mark)

EVALUATION

<i>Sl. No.</i>	<i>Program Name</i>	<i>Date</i>	<i>Lab Work (5M)</i>	<i>Viva Voice (5M)</i>	<i>Total (10M)</i>	<i>Sign</i>
1.	1.1	<i>Blink the LED_BUILTIN using a blink LED program from examples.</i>				
	1.2	<i>Display “Hello World “Using the Serial monitor tool.</i>				
	1.3	<i>Write a program to blink 2 alternate LEDs out of 4 LEDs with delay of 3 and 5 seconds respectively</i>				
2.	2.1	<i>Write a program to display Numbers in the increasing and decreasing order from 1 to 15 and vice versa in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value (15), start counting down to 1, then again begin counting to 15, then start from 1. Whenever it comes to the Max or Min value, wait for 2 seconds.</i>				
	2.2	<i>Write a program to display odd, even numbers in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value, wait 2 seconds and start from the minimum. (Example: 0001, 0010, 0011, ..., 1111, 1110, 1101, ..., 0001,0010,0011, ...)</i>				
3.	3.1	<i>Write an assembly function that just returns 200 back to the C program and write an assembly function that returns sum of three constants 10, 20 and 30, and returns the result.</i>				

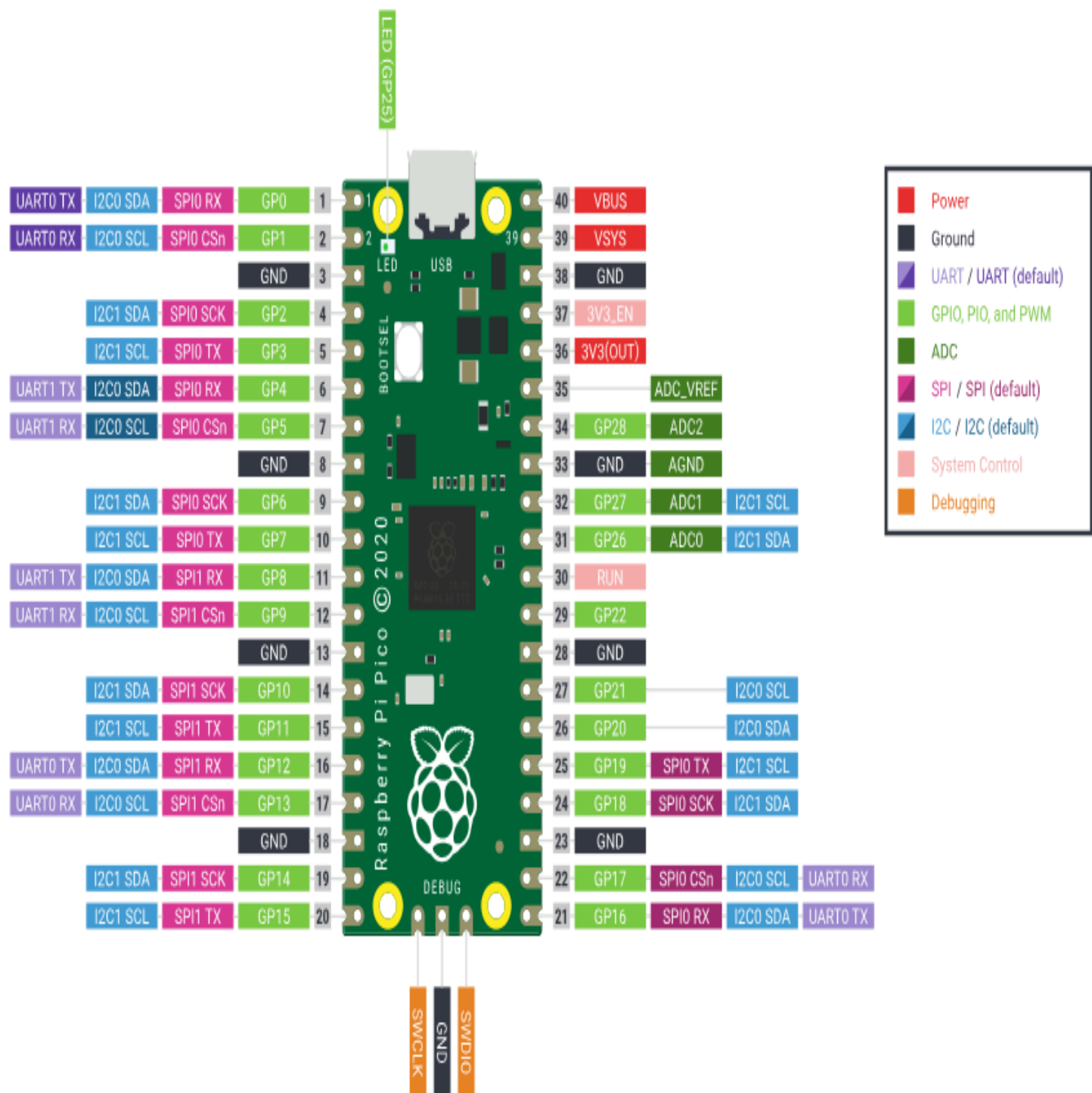
	3.2	<i>Write an assembly function that returns sum of four constants 10, 20, 30 and 40, and returns the result.</i>					
	3.3	<i>Write an assembly function that returns sum of five constants 10, 20, 30, 40 and 50, and returns the result.</i>					
4.	4.1	<i>Write an ARM assembly function that takes three constants as inputs from C program, perform ADD operation, and returns the result to the C program.</i>					
	4.2	<i>Write an ARM assembly function that takes two input parameters, performs an AND, OR logic operation on it, and returns the result to a calling C program.</i>					
	4.3	<i>Write an ARM assembly function that takes one input parameter, performs a bitwise NOT operation on it, and returns the result to a calling C program.</i>					
5.	5.1	<i>Write an ARM assembly function that performs 2 times logical shift right operation on one parameter and returns the result to a calling C program.</i>					
	5.2	<i>Write an ARM assembly function that performs 2 times logical shift right and 4 times logical shift left operation on one parameter and returns the result to a calling C program</i>					

6.		<i>Write an ARM assembly function that performs 8 times arithmetic shift right operation on one signed (Negative number) parameter and returns the result to a calling C program.</i>					
7.	7.1	<i>Verify the De-morgans theorem with two inputs. If the result is TRUE, print both the results and “De-morgans theorem is verified”, else, print both the results and “De-morgans theorem is not verified”.</i>					
	7.2	<i>Perform EX-OR operation with two inputs. If the result is TRUE, print both the results and “EX-OR logic expression is verified”, else, print both the results and “EX-OR logic expression is not verified”.</i>					
8.	8.1	<i>Write ARM assembly functions to take two unsigned parameters and perform an ADD operation.</i>					
	8.2	<i>Write ARM assembly functions to Read the flag bits from the PSR. Return both outputs to the main function.</i>					
9	9.1	<i>Write a program for the Raspberry Pi Pico to transmit and receive data through serial communication (UART) port.</i> <i>Transmitting and Receiving Board (RP2040): UART0, GP0 (UART0 Tx), GP1 (UART0 Rx)</i>					

10	10.1	<i>Write a program to control one slave device from one master device using serial communication through the I2C port with the following specifications.</i> <i>Master Slave Board (RP2040): I2C0, GPIO pins: GP0 (I2C0 SDA), GP1 (I2C0 SCL)</i>					
		Total (100)					
		Total (5)					

RECORD	Max – 5	
TEST		
Write-up with Correct Code	Max - 05	
Code Execution and Hardware Interfacing	Max - 10	
VIVA	Max - 05	
TOTAL	Max - 20(Scale Down to 10)	
TOTAL LAB MARKS	MAX-15	
Signature of the faculty		

Raspberry Pi Pico – RP2040 Pin Description



Experiment 1

1.1 Blink the LED_BUILTIN using a blink LED program,

1.2 Display “Hello World “Using the Serial monitor tool,

1.3 Write a program to blink 2 alternate LEDs out of 4 LEDs with delay of 3 and 5 seconds respectively.

Objectives:

1. Turn a Raspberry Pi Pico (RP2040) LED_BUILTIN **on** and **off** every second. Change the delay and see the difference in blinking speed.
2. Learn how to use a Serial monitor.
3. Write a program to blink LED using Raspberry Pi Pico (RP2040) GPIO pins.

Components required:

Sl No	Description	Specification	Quantity
1 2 3	Arduino IDE software	Software	-
4 5	Raspberry Pi Pico (RP2040) board	RP2040	1
6	Micro USB cable	1mtr	1
	LED	5mm	4
	Resistor	1K Ω	4
	Breadboard	-	1

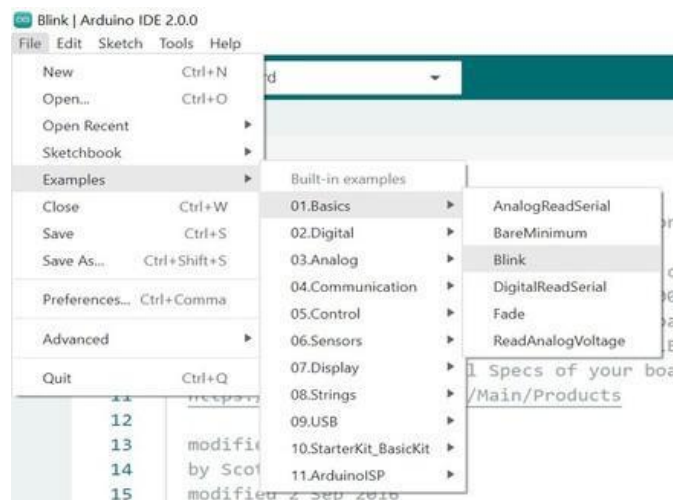
Experiment 1.1:

BlinktheLED_BUILTIN using a blink LED program.

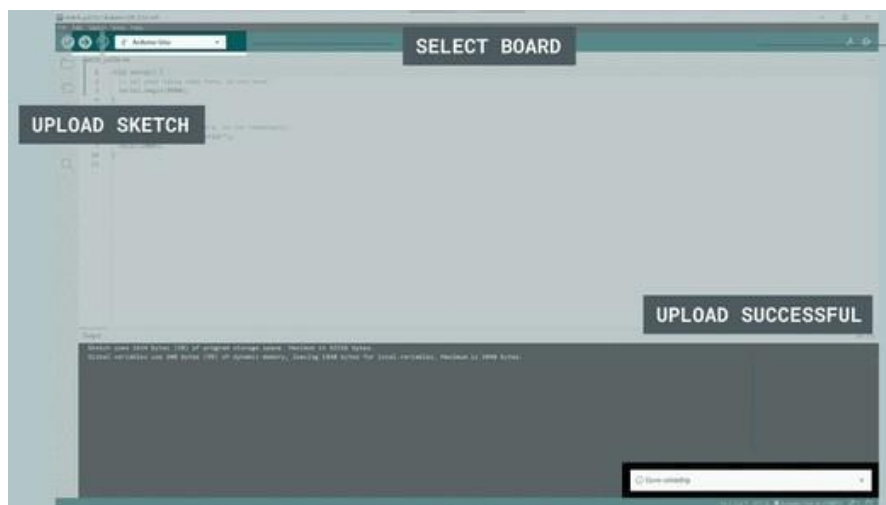
```
void setup() {  
  pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH);  
  delay(500);  
  digitalWrite(LED_BUILTIN, LOW);  
  delay(500);  
}
```

Procedure:

1. ConnecttheRaspberry Pi Pico board to the PC USB port using the micro-USB cable.
2. OpentheArduino IDE 2.0.4.
3. Clickonthe**Select Board** in Arduino IDE.
4. Clickontheoption shown “**Select other board and port ...**”.
5. Selecttheboard and the port in the **Select other board and board** window.
6. In theArduino IDE, click the **File** menu on the top menu bar.
7. Followthebelow steps to fetch the blink LED program from **examples**.



8. Create one folder for the ESM course lab and save this program in the folder.
9. To compile the program, press the verify button (Tick mark) in the left corner.
10. After compilation is completed, press the Upload button (->) at the top left corner.



11. Once the program is uploaded into the board, the program image is saved in flash memory.
12. This makes the built-in LED (Red) blink at 1000 milliseconds.
13. Now upload the program with different delays (i.e., 500 milliseconds / 300 milliseconds / 100 milliseconds) and see the difference in blinking speed.

Experiment 1.2:

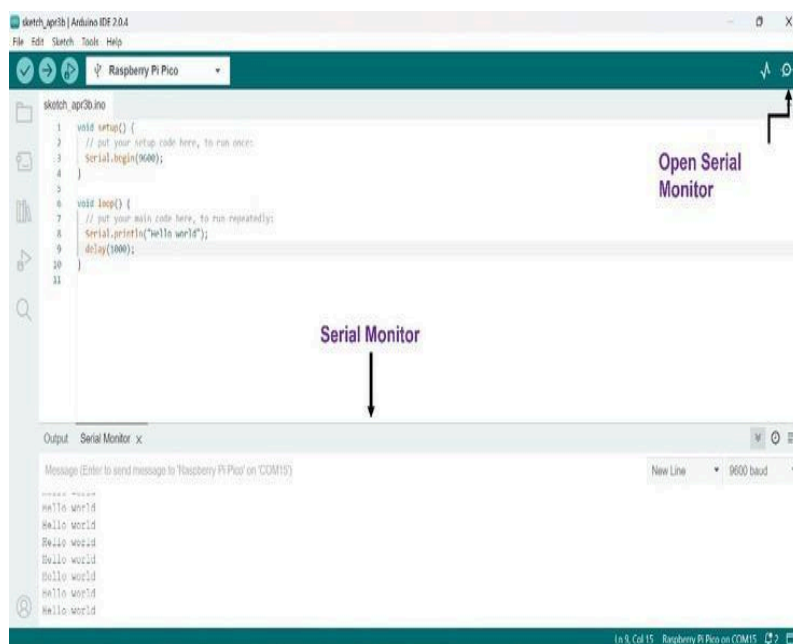
Display "Hello World" Using the Serial monitor tool.

```
void setup() {
  Serial.begin(9600);
}
void loop() {
  Serial.println("Hello world!");
  delay(1000);
}
```

Procedure:

- 1.
2. Click on the **File** menu on the top menu bar and create a **new sketch** by clicking on the **new sketch** option in the **File** menu.
3. Set a baud rate, by writing **Serial.begin(9600);** in void setup.
(Note: Here, the 9600 represents the baud rate, the maximum bits per second that can be transferred.)
4. Print the **Hello world** by writing a **Serial.println("Hello world");** in the void loop, give a 1-second delay.
5. When it has finished uploading, click on the **serial monitor** button at the IDE's top right corner. This will launch the serial monitor at the bottom of the editor, replacing the console section.
6. In the **serial monitor**, adjust the baud rate to **9600** at the bottom right corner.

7. The text ***Hello world*** is now printed every a 1-second, as shown below.



8. Print your name and USN in the serial monitor with different baud rates.

Experiment 1.3:

Write a program to blink 4 LEDs at a time using 4, 6, 9 and 22 GPIO pins.

```
#define TRUE 1
#define FALSE 0
const int numLEDs = 4;
const int GPIO_Pins[] = {4, 6, 9, 22};
const int blinkRate = 1000;
static bool flag = TRUE;
void setup() {
    //put your setup code here, to run once:
    for(int led = 0; led < numLEDs; led++)
    {
        pinMode(GPIO_Pins[led], OUTPUT);
    }
}
void loop() {
    //put your main code here, to run repeatedly:
    for(int led = 0; led < numLEDs; led++)
    {
        digitalWrite(GPIO_Pins[led], !digitalRead(GPIO_Pins[led]));
        flag = !flag;
        void controlLEDs(int led, bool state)
        {
            if(state == TRUE)
                digitalWrite(GPIO_Pins[led], HIGH);
            else
                digitalWrite(GPIO_Pins[led], LOW);
        }
    }
}
```

Activity 1.1

1. Write a program to blink 4 LEDs at a different time using 19, 17, 2 and 12 GPIO pins. (i.e., 1st two LEDs have to be ON when another two LEDs are OFF.)
2. Write a program to blink 2 alternate LEDs out of 4 LEDs with delay of 3 and 5 seconds respectively.
3. Write a program for Raspberry Pi Pico board that listens for input from the serial monitor and turns ON one of four LEDs when a specific number is entered by using below function syntax.
 - A. ***Serial.available()*** Get the number of bytes available for reading from the serial port. This is data that's already arrived and stored in the serial receive buffer (Which holds 64bytes).
 - B. ***If(Serial.available() > 0)*** It will only send data when the received data is greater than zero.
 - C. ***Serial.parseInt()*** Looks for the next valid integer in the incoming serial. The function terminates if it times out (i.e., If no valid digits were read when the time-out occurs, 0 is returned

Experiment 2

2.1 Write a program to display Numbers in the increasing and decreasing order from 1 to 15 and vice versa in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value (15), start counting down to 1, then again begin counting up to 15, then start from 1. Whenever it comes to the Max or Min value, wait for 2 seconds.

2.2 Write a program to display odd, even numbers in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value, wait 2 seconds and start from the minimum.

Objectives:

1. Learn to write a program to blink 4 LEDs in different ways.
2. Learn to display binary count using 4 LEDs.

Components required:

Sl No	Description	Specification	Quantity
1 2 3	Arduino IDE software	Software	-
4 5 6	Raspberry Pi Pico (RP2040) board	RP2040	1
	Micro USB cable	1mtr	1
	LED	5mm	4
	Resistor	1K Ω	4
	Breadboard	-	1

Experiment 2.1: Write a program to display the numbers in the *increasing and decreasing order* from 1 to 15 and vice versa in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value (15), start counting down to 1, then again begin counting up to 15, then start from 1. Whenever it comes to the Max or Min value, wait for 2 seconds. (Example: 0001, 0010, 0011, ..., 1111, 1110, 1101, ..., 0001, 0010, 0011, ...)

```
// Define the pins for the LEDs
const int ledPin1 = 2;
const int ledPin2 = 3;
const int ledPin3 = 4;
const int ledPin4 = 5;

void setup() {
  // Initialize the LED pins as outputs
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  pinMode(ledPin3, OUTPUT);
  pinMode(ledPin4, OUTPUT);
}

void loop() {
  for (int i = 1; i <= 15; ++i) { // Counting up from 1 to 15
    displayBinary(i);
    delayWithCheck(i == 15); // Delay with longer pause if i is 15
  }

  // Counting down from 15 to 1
  for (int i = 15; i >= 1; --i) {
```

```

    displayBinary(i);
    delayWithCheck(i == 1); // Delay with longer pause if i is 1
}
}

// Function to display the binary representation of a number on LEDs
void displayBinary(int number) {
    digitalWrite(ledPin1, number & 0x01);

    digitalWrite(ledPin2, (number >> 1) & 0x01);
    digitalWrite(ledPin3, (number >> 2) & 0x01);
    digitalWrite(ledPin4, (number >> 3) & 0x01);
    delay(1000); // Delay of 1 second
}

//Function to add an extra delay if flag is true (for max/min value pause)
void delayWithCheck(bool flag) {
    if(flag) {

        delay(2000); // Pause for 2 seconds at max or min value
    }else {

        delay(1000);
    }
}
}

```

Experiment 2.2:

Write a program to display odd, even numbers in binary form, using 4 LEDs, with a delay of 1 sec. When it reaches the maximum value, wait 2 seconds and start from the minimum.

```

#define TRUE 1
#define FALSE 0

const int numLEDs = 4;
static int arr[5];
const int GPIO_Pins[] = {5, 4, 3, 2};
void setup() {

    // put your setup code here, to run once:
    for (int led = 0; led < numLEDs; led++){
        pinMode(GPIO_Pins[led], OUTPUT);
    }
}

void loop() {

    // put your main code here, to run repeatedly:
    for (int i = 1; i <= 15; i+=2)
// for (int i = 0; i <= 15; i+=1) for even//

```

```

{
if (i % 2 != 0) {
    binaryLEDs(i);
}
delay(1000);
}
delay(2000);
}
void binaryLEDs(int num){
for (int i = 0; i < 4; i++) {
    arr[i] = num %
    2; num = num /
    2; if (arr[i] == 1)
        digitalWrite(GPIO_Pins[i], HIGH);
    else
        digitalWrite(GPIO_Pins[i], LOW);
}
}
}

```

Activity 2.1:

1. Write a program to blink 4 LEDs simultaneously using an array with a delay of 1 sec.
2. Write a program to display blinking 4 LEDs in one direction with a delay of 1 sec.
3. Write a program to display blinking 4 LEDs from left to right and vice versa with a delay of 1 sec.

Activity 2.2:

Write a program to display the numbers in **increasing order** from 0 to 15 in a 4-bit display with a delay of 1 sec. While showing a number, show its complement, too, with a delay of 1 sec. Once it reaches the maximum value, wait 2 seconds, and continue from 0

(Example: 0000, 1111, 0001, 1110, 0010, 1101, ...)

Experiment 3

- C.1** Write an assembly function that just returns 200 back to the C program and write an assembly function that returns sum of three constants 10, 20 and 30, and returns the result.
- C.2** Write an assembly function that returns sum of four constants 10, 20, 30 and 40, and returns the result.
- C.3** Write an assembly function that returns sum of five constants 10, 20, 30, 40 and 50, and returns the result.

Objectives:

Learn the execution of assembly programming along with C program.

Components required:

Sl No	Description	Specification	Quantity
1 2 3	Raspberry Pi Pico board	RP2040	1
	Micro USB cable	1mtr	1
	Arduino IDE	Software	-

Note:

This needs to be built with the board package of **MbedOS**.

Board and port selection process:

- Click on the option shown “**Select other board and port ...**”.
- In the search window type “**Raspberry**”.
- Then select the Option appearing below
- “Raspberry Pi Pico - Arduino MbedOS RP2040 boards”.
- Select a **USB port** and upload the program.

If you try to upload this program with other Pico board choices, the board will become unresponsive. **CAUTION1:**

Whenever the program becomes unresponsive because of wrong assembly programs which may hang the board due to some errors in your program, press the BOOT SEL button and connect the board to the PC USB port using a micro- USB cable. Release the button and select Board (“**Raspberry Pi Pico - Arduino MbedOS RP2040 boards**”) and port (“**UF2 Board**”). Upload the program.

CAUTION2: DO NOT UPLOAD this program with

“Raspberry Pi Pico - Raspberry Pi Pico / RP2040”

Experiment 3.1:

Write an assembly function that just returns 200 back to the C program and write an assembly function that returns sum of three constants 10, 20 and 30, and returns the result.

```
#include <stdio.h>
#include <stdlib.h>
void myPrint(char * fnName, unsigned int result);
extern "C" {
    unsigned int my_asm_fn(void);
}
void setup() {
    // put your setup code here, to run once:
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
    // put your main code here, to run repeatedly:
    unsigned int asm_ret_val;
```

```

asm_ret_val = my_asm_fn();
myPrint("my_asm_fn()",asm_ret_val);
digitalWrite(LED_BUILTIN,HIGH);
delay(1000);
digitalWrite(LED_BUILTIN,LOW);
delay(1000);
}

void myPrint(char * fnName, unsigned int result)
{
  Serial.print("The value returned from the assembly fn ");
  Serial.print(fnName);
  Serial.print(" : ");
  Serial.println(result);
}

```

C program for Adding 3 constants

```

#include <stdio.h>
#include <stdlib.h>

void myPrint(char * fnName, unsigned int result);
extern "C" {
  unsigned int my_asm_add_two_constants(void);
}

void setup() {
  //put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(LED_BUILTIN,OUTPUT);
}

void loop() {
  //put your main code here, to run repeatedly:
  unsigned int asm_ret_val;
  asm_ret_val = my_asm_add_two_constants();
  myPrint("my_asm_add_two_constants()",asm_ret_val);
  digitalWrite(LED_BUILTIN,HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN,LOW);
  delay(1000);
}

void myPrint(char * fnName, unsigned int result)
{
  Serial.print("The value returned from the assembly fn ");
  Serial.print(fnName);
  Serial.print(" : ");
  Serial.println(result);
}

```

Asm Program

```

.global my_asm_fn

.t ext
my_asm_fn:
  movr0, #200
  bxlr
.end

.global my_asm_add_three_constants
.t ext

```

```

my_asm_add_three_constants:
    mov r3, #30
    mov r2, #20
    mov r1, #10
    add r3, r3, r2
    add r0, r1, r3
    bx lr
.end

```

Experiment 3.2:

Write an assembly function that returns sum of four constants 10, 20, 30 and 40, and returns the result.

```

#include <stdio.h>
#include <stdlib.h>

void myPrint(char * fnName, unsigned int result);
extern "C" {
    unsigned int my_asm_add_four_constants(void);
}

void setup() {
    //put your setup code here, to run once:
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
    //put your main code here, to run repeatedly:
    unsigned int asm_ret_val;
    asm_ret_val = my_asm_add_four_constants();
    myPrint("my_asm_add_four_constants()", asm_ret_val);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}

void myPrint(char * fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program

```

.global my_asm_add_four_constants

.text
my_asm_add_four_constants:
    mov r4, #40
    mov r3, #30
    mov r2, #20
    mov r1, #10
    add r4, r4, r3
    add r4, r4, r2
    add r0, r1, r4
    bx lr
.end

```

Experiment 3.3:

Write an assembly function that returns sum of five constants 10, 20, 30, 40 and 50, and returns the result.

```
#include <stdio.h>
#include <stdlib.h>
void myPrint(char * fnName, unsigned int result);
extern "C"{
    unsigned int my_asm_add_five_constants(void);
}
void setup() {
    //put your setup code here, to run once:
    Serial.begin(9600);
    pinMode(LED_BUILTIN,OUTPUT);
}
void loop() {
    //put your main code here, to run repeatedly:
    unsigned int asm_ret_val;
    asm_ret_val = my_asm_add_five_constants();
    myPrint("my_asm_add_five_constants()",asm_ret_val);
    digitalWrite(LED_BUILTIN,HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN,LOW);
    delay(1000);
}
void myPrint(char * fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}
```

Asm Program

```
.global my_asm_add_five_constants
.text
my_asm_add_five_constants:
    mov r5, #50
    mov r4, #40
    mov r3, #30
    mov r2, #20
    mov r1, #10
    add r5, r5, r4
    add r5,r5,r3
    add r5, r5, r2
    add r0,r1,r5
    bx lr
.end
```

Activity 3.1

Write an assembly function that just returns 150 back to the C program and write an assembly function that returns sum of three constants 20,30 and 50, and returns the result

Activity 3.2:

Write an assembly function that returns sum of four constants 40, 20, 50 and 60, and returns the result.

Activity 3.3:

Write an assembly function that returns sum of five constants 5, 10, 20, 40 and 50, and returns the result.

Experiment 4

1. Write an ARM assembly function that takes three constants as inputs from C program, perform ADD operation, and returns the result to the C program.
2. Write an ARM assembly function that takes two input parameters, performs an AND logic operation on it, and returns the result to a calling C program.
3. Write an ARM assembly function that takes one input parameter, performs a bitwise NOT operation on it, and returns the result to a calling C program.

Objectives:

- Learn how to implement bitwise operations in assembly language along with the C program.

Components required:

SlNo	Description	Specification	Quantity
1	Raspberry Pi Pico board RP2040		1
2	Micro USB cable	1mtr	1
3	Arduino IDE	Software	-

Note:

This needs to be built with the board package of **MbedOS**.

Board and port selection process:

- Click on the option shown “**Select other board and port ...**”.
- In the search window type “**Raspberry**”.
- Then select the Option appearing below

“**Raspberry Pi Pico - Arduino MbedOS RP2040 boards**”.

- Select a **USB port** and upload the program.

If you are able to build it using the “**Raspberry Pi Pico-Raspberry Pi Pico/RP2040**”, continue and you are free to upload it on to the board.

If you try to upload this program with a different Pico board choices than what is already running on the board, the board will become unresponsive.

Remember that when you change this package, from what is running on the board, you will have Upload error and you need to follow the below steps to change the package from one to the other:

- Remove the board from the USB port
- Keep pressing the BOOTSEL button while connecting the board to the USB port of your laptop

- In the Arduino IDE, click the **Tools** menu on the top menu bar, select Board and port (“**UF2 Board**”). Upload the program.
- Upload the code even if the COM port is not recognized by the Arduino IDE
- In case of upload failure, now disconnect the board from the USB port and connect it back
- Now, Arduino IDE should recognize the board and COM port number should appear and try uploading the software now, it should go through successfully.

Experiment 4.1:

Write an ARM assembly function that takes three constants as inputs from the C program, performs ADD operation and returns the result to the C program.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
extern "C"
```

```
{
```

```
    unsigned int lab_4(unsigned int, unsigned int, unsigned int);
```

```
}
```

```
void setup()
```

```
{
```

```
    Serial.begin(9600);
```

```
    pinMode(LED_BUILTIN, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
    unsigned int result;
```

```
    result = lab_4(10, 20, 30);
```

```
    myPrint("Result of ASM program: ", result);
```

```
    digitalWrite(LED_BUILTIN, HIGH);
```

```
    delay(1000);
```

```
    digitalWrite(LED_BUILTIN, LOW);
```

```
    delay(1000);
```

```
}
```

```

void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program

```

.global lab_4
.text    lab_4:
add  R0,R0,R1
add R0,R0,R2
bx lr .end

```

Experiment 4.2:

Write an ARM assembly function that takes two input parameters, performs an AND logic operation on it, and returns the result to a calling C program.

```

#include <stdio.h>
#include <stdlib.h>

extern "C"
{
    unsigned int lab_4(unsigned int, unsigned int);
}

void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}

```

```

void loop()
{
    unsigned int result;
    result = lab_4(10, 20);

    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}

```

```

void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program

```

.global lab_4
.text
lab_4:
    and R0,R0,R1
    bx lr
.end

```

Experiment 4.3:

Write an ARM assembly function that takes one input parameter, performs a bitwise NOT operation on it, and returns the result to a calling C program.

```

#include <stdio.h>
#include <stdlib.h>

```

```

extern "C"
{
    unsigned int lab_4(unsigned int);
}

```

```
}
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  pinMode(LED_BUILTIN, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
  unsigned int result;
```

```
  result = lab_4(10);
```

```
  myPrint("Result of ASM program: ", result);
```

```
  digitalWrite(LED_BUILTIN, HIGH);
```

```
  delay(1000);
```

```
  digitalWrite(LED_BUILTIN, LOW);
```

```
  delay(1000);
```

```
}
```

```
void myPrint(char *fnName, unsigned int result)
```

```
{
```

```
  Serial.print("The value returned from the assembly fn ");
```

```
  Serial.print(fnName);
```

```
  Serial.print(" : ");
```

```
  Serial.println(result);
```

```
}
```

Asm Program

```
.global lab_4
```

```
.text
```

```
lab_4:
```

```
mvn R0,R0
```

```
bx lr
```

```
.end
```

Activity 4.1:

Write an ARM assembly function that takes two input parameters, performs an OR logic operation on it, and returns the result to a calling C program

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
extern "C"
```

```
{  
    unsigned int lab_4(unsigned int, unsigned int);  
}
```

```
void setup()
```

```
{  
    Serial.begin(9600);  
    pinMode(LED_BUILTIN, OUTPUT);  
}
```

```
void loop()
```

```
{  
    unsigned int result;  
    result = lab_4(10, 20);  
    myPrint("Result of ASM program: ", result);  
    digitalWrite(LED_BUILTIN, HIGH);  
    delay(1000);  
    digitalWrite(LED_BUILTIN, LOW);  
    delay(1000);  
}
```

```
void myPrint(char *fnName, unsigned int result)
```

```
{  
    Serial.print("The value returned from the assembly fn ");  
    Serial.print(fnName);  
    Serial.print(" : ");
```

```
Serial.println(result);  
}
```

Asm Program

```
.global lab_4  
.text  
lab_4:  
orr R0,R0,R1  
bx lr  
.end
```

Activity 4.2:

Write an ARM assembly function that takes two input parameters, performs an Exclusive OR logic operation on it, and returns the result to a calling C program.

```
#include <stdio.h>  
#include <stdlib.h>  
  
extern "C"  
{  
    unsigned int lab_4(unsigned int, unsigned int);  
}  
  
void setup()  
{  
    Serial.begin(9600);  
    pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop()  
{  
    unsigned int result;  
    result = lab_4(10, 20);  
    myPrint("Result of ASM program: ", result);  
    digitalWrite(LED_BUILTIN, HIGH);  
    delay(1000);  
}
```

```

digitalWrite(LED_BUILTIN, LOW);
delay(1000);
}

void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program

```

.global lab_4
.text
lab_4:
eor R0,R0,R1
bx lr
.end

```

Activity 4.3:

Write an ARM assembly functions for the following expressions.

- $(val_a \gg 4) \& val_b$

Parameters: val_a, val_b, n=4 (unsigned int)

- $(val_a \ll 8) | (\sim val_b)$

Parameters: val_a, val_b, n=8 (unsigned int)

- $(sVal_a \gg 2) \& (\sim sVal_b)$

Parameters: sVal_a, sVal_b, n=2 (signed int)

- $(\sim sVal_a \gg 5) \wedge (sVal_b)$

Parameters: sVal_a, sVal_b, n=5 (signed int)

i. (val_a >> 4) & val_b

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
extern "C"
```

```
{
```

```
    unsigned int lab_4(unsigned int, unsigned int);
```

```
}
```

```
void setup()
```

```
{
```

```
    Serial.begin(9600);
```

```
    pinMode(LED_BUILTIN, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
    unsigned int result;
```

```
    result = lab_4(10, 20);
```

```
    myPrint("Result of ASM program: ", result);
```

```
    digitalWrite(LED_BUILTIN, HIGH);
```

```
    delay(1000);
```

```
    digitalWrite(LED_BUILTIN, LOW);
```

```
    delay(1000);
```

```
}
```

```
void myPrint(char *fnName, unsigned int result)
```

```
{
```

```
    Serial.print("The value returned from the assembly fn ");
```

```
    Serial.print(fnName);
```

```
    Serial.print(" : ");
```

```
    Serial.println(result);
```

```
}
```

Asm Program

```
.global lab_4
.text
lab_4:
mov r2,#4
lsr r0,r0,r2
and r0,r0,r1
bx lr
.end
```

ii. $(val_a \ll 8) | (\sim val_b)$

```
#include <stdio.h>
#include <stdlib.h>
```

```
extern "C"
{
    unsigned int lab_4(unsigned int, unsigned int);
}
```

```
void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}
```

```
void loop()
{
    unsigned int result;
    result = lab_4(10, 20);
    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
}
```

```
digitalWrite(LED_BUILTIN, LOW);
delay(1000);
}
```

```
void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}
```

Asm Program

```
.global lab_4
.text
lab_4:
mov r2,#8
lsl r0,r0,r2
mvn r3,r1
orr
r0,r0,r3 bx
lr
.end
```

iii. (sVal_a >> 2) & (~sVal_b)

```
#include <stdio.h>
#include <stdlib.h>
```

```
extern "C"
{
    unsigned int lab_4(unsigned int, unsigned int);
}
```

```
void setup()
{
    Serial.begin(9600);
```

```

    pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
    unsigned int result;
    result = lab_4(10, 20);
    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}

void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program

```

.global lab_4
.text
lab_4:
    mov r2,#8
    asr
    r0,r0,r2
    mvn r3,r1
    and r0,r0,r3
    bx lr
.end

```

iv. $(\sim sVal_a \gg 5) \wedge (sVal_b)$

```

#include <stdio.h>

```

```
#include <stdlib.h>
```

```
extern "C"
```

```
{  
    unsigned int lab_4(unsigned int, unsigned int);  
}
```

```
void setup()
```

```
{  
    Serial.begin(9600);  
    pinMode(LED_BUILTIN, OUTPUT);  
}
```

```
void loop()
```

```
{  
    unsigned int result;  
    result = lab_4(10, 20);  
    myPrint("Result of ASM program: ", result);  
    digitalWrite(LED_BUILTIN, HIGH);  
    delay(1000);  
    digitalWrite(LED_BUILTIN, LOW);  
    delay(1000);  
}
```

```
void myPrint(char *fnName, unsigned int result)
```

```
{  
    Serial.print("The value returned from the assembly fn ");  
    Serial.print(fnName);  
    Serial.print(" : ");  
    Serial.println(result);  
}
```

Asm Program:

```
.global lab_4
```

```
.text
```

```
lab_4:
```

```
mov r2,#5
```

```
lsr r0,r0,r2
```

```
eor r0,r0,r1
bx lr
.end
```

Experiment 5

5.1 Write an ARM assembly function that performs 2 times logical shift right operation on one parameter and returns the result to a calling C program.

5.2 Write an ARM assembly function that performs 2 times logical shift right and 4 times logical shiftleft operationon one parameter andreturns the result to a calling C program

Objectives:

Learnhowtoimplementbitwiseoperationsinassemblylanguage along with the C program.

Components required:

SI No	Description	Specification	Quantity
1 2 3	Raspberry Pi Pico board	RP2040	1
	Micro USB cable	1mtr	1
	Arduino IDE	Software	-

Note:

This needs to be built with the board package of **MbedOS**.

Board and port selection process:

- Click on the option shown “**Select other board and port ...**”.
- In the search window type “**Raspberry**”.
- Then select the Option appearing below
“**Raspberry Pi Pico - Arduino MbedOS RP2040 boards**”.
- Select a **USB port** and upload the program.

If you are able to build it using the “**Raspberry Pi Pico-Raspberry Pi Pico/RP2040**”, continue and you are free to upload it on to the board.

If you try to upload this program with a different Pico board choices than what is already running on the board, the board will become unresponsive.

Remember that when you change this package, from what is running on the board, you will have Upload error and you need to follow the below steps to change the package from one to the other:

- Remove the board from the USB port
- Keep pressing the BOOTSEL button while connecting the board to the USB port of your laptop
- In the Arduino IDE, click the **Tools** menu on the top menu bar, select Board and port (“**UF2 Board**”).
- Upload the program.
- Upload the code even if the COM port is not recognized by the Arduino IDE
- In case of upload failure, now disconnect the board from the USB port and connect it back
Now, Arduino IDE should recognize the board and COM port number should appear and try uploading the software now, it should go through successfully.

Experiment 5.1:

Write an ARM assembly function that performs 2 times logical shift right operation on one parameter and returns the result to a calling C

```
#include <stdio.h>
#include
<stdlib.h> extern
"C"
```

```

{
    unsigned int lab_5_1(unsigned int, unsigned int, unsigned int);
}

void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
    unsigned int result;
    result = lab_5_1(10);
    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}

void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program:

```

.global lab_5_1
.text
lab_5_1:
    LSR r0, r0, #2
    bx lr
.end

```

Experiment 5.2:

Write an ARM assembly function that performs 4 times logical shift left operation on one parameter and returns the result to a calling C program.

```

#include <stdio.h>

```

```

#include <stdlib.h>

extern "C"

{
    unsigned int lab_5_2(unsigned int);
}

void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
    unsigned int result;
    result = lab_5_2(10);
    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}

void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program:

```

.global lab_5_2

.text

lab_5_2:
    LSL r0, r0, #4
    bx lr
.end

```

Activity

5.1

Write an ARM assembly functions for the following expressions.

- $(val_a \gg 4) \& val_b$ Parameters: val_a, val_b, n=4 (unsigned int)
- $(val_a \ll 8) | (\sim val_b)$ Parameters: val_a, val_b, n=8 (unsigned int)
- $(sVal_a \gg 2) \& (\sim sVal_b)$ Parameters: sVal_a, sVal_b, n=2 (signed int)\
- $(\sim sVal_a \gg 5) ^ (sVal_b)$ Parameters: sVal_a, sVal_b, n=5 (signed int)

```

    .    (val_a >> 4) &
val_b #include <stdio.h>
#include <stdlib.h>
extern "C"
{
    unsigned int lab_5(unsigned int, unsigned int);
}
void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}
void loop()
{
    unsigned int result;
    result = lab_5(10, 20);
    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}
void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program:

```
.global lab_5
.text
lab_5:
mov r2,#4
lsr r0,r0,r2
and r0,r0,r1
bx lr
.end
```

```
ii.      (val_a << 8) |
(~val_b) #include <stdio.h>
#include <stdlib.h>
extern "C"
{
    unsigned int lab_5(unsigned int, unsigned int);
}
void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}
void loop()
{
    unsigned int result;
    result = lab_5(10, 20);
    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}
void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
```

```

Serial.print(fnName);

Serial.print(" : ");

Serial.println(result);
}

```

Asm Program:

```

.global lab_5

.text

lab_5:
mov r2,#8
lsl r0,r0,r2
mvn r3,r1
orr
r0,r0,r3 bx
lr
.end

```

```

iii. (sVal_a >> 2) &
(~sVal_b) #include <stdio.h>
#include <stdlib.h>
extern "C"
{
    unsigned int lab_5(unsigned int, unsigned int);
}

void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
    unsigned int result;
    result = lab_5(10, 20);
    myPrint("Result of ASM program: ", result);
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
}

```

```

    delay(1000);
}
void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program:

```

.global    lab_5
.text lab_5: mov
r2,#8      asr
r0,r0,r2    mvn
r3,r1,r3    and
r0,r0,r3    bx    lr
.end

```

```

iv.    (~sVal_a >> 5)
^(sVal_b) #include <stdio.h>
#include <stdlib.h>
extern "C"
{
    unsigned int lab_5(unsigned int, unsigned int);
}
void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}
void loop()
{

```

```

unsigned int result;

result = lab_5(10, 20);

myPrint("Result of ASM program: ", result);

digitalWrite(LED_BUILTIN, HIGH);

delay(1000);

digitalWrite(LED_BUILTIN, LOW);

delay(1000);

}

void myPrint(char *fnName, unsigned int result)
{
    Serial.print("The value returned from the assembly fn ");
    Serial.print(fnName);
    Serial.print(" : ");
    Serial.println(result);
}

```

Asm Program:

```

.global lab_5

.text

lab_5:
    mov r2,#5
    lsr r0,r0,r2
    eor r0,r0,r1
    bx lr

.end

```

Experiment 6

Write an ARM assembly function that performs 8 times arithmetic shift right operation on one signed (Negative number) parameter and returns the result to a calling C program.

Objectives:

- Learn how to implement bitwise operations in assembly language along with the C program.

Components required:

	SI No	1	2	3	Description	Raspberry		S
					Pi Pico board	Micro USB cable		p
					Arduino IDE			e
								c

i
f
i
c
a
t
i
o
n

Q
u
a
n
t
i
t
y
R
P
2
0
4
0

1

1mtr
Software

1
-

Note:

This needs to be built with the board package of *MBedOS*.
Board and port selection process:

- Click on the option shown “*Select other board and port ...*”.
- In the search window type “*Raspberry*”.
- Then select the Option appearing below
“*Raspberry Pi Pico - Arduino MbedOS RP2040 boards*”.
- Select a *USB port* and upload the program.

If you are able to build it using the “*Raspberry Pi Pico-Raspberry Pi Pico/RP2040*”, continue and you are free to upload it on to the board.

If you try to upload this program with a different Pico board choices than what is already running on the board, the board will become unresponsive.

Experiment No-6

Write an ARM assembly function that performs 8 times arithmetic shift right operation on one signed (Negative number) parameter and returns the result to a calling C program.

```
include <stdio.h>
extern "C"
{
    signed int expt(signed int val_1)
}
void setup()
```

```

{
  Serial.begin(9600);
  pinMode(LED_BUILTIN,OUTPUT);
}
void loop()
{
  signed int val_1;
  val_1=-256;
  signed int result;
  result=expt(val_1);
  Serial.println(result,HEX);
  delay(1000);
  digitalWrite(LED_BUILTIN,HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN,LOW);
  delay(1000);
}

```

Asm Program:

```

.global expt
.text
expt:
asr r0,r0,#8
bx lr
.end

```

Activity Program:

Write an ARM assembly function that performs 8 times arithmetic shift left operation on one signed (Negative number) parameter and returns the result to a calling C program.

```
#include <stdio.h>
```

```
extern "C"
```

```

{

  signed int expt(signed int val_1);
}

void setup()
{

```

```
Serial.begin(9600);  
pinMode(LED_BUILTIN,OUTPUT);  
}  
void loop()  
{  
  signed int val_1;  
  val_1=-256;  
  signed int result;  
  result=expt(val_1);  
  Serial.println(result,HEX);  
  delay(1000);  
  digitalWrite(LED_BUILTIN,HIGH);  
  delay(1000);  
  digitalWrite(LED_BUILTIN,LOW);  
  delay(1000);  
}
```

Asm Program:

```
.global expt  
.text  
expt:  
lsl r0,r0,#8  
bx lr  
.end
```

Experiment 7

7.1 Verify the De-morgans theorem with two inputs. If the result is TRUE, print both the results and “De-morgans theorem is verified”, else, print both the results and “De-morgans theorem is not verified”.

7.2 Perform EX-OR operation with two inputs. If the result is TRUE, print both the results and “EX-OR logic expression is verified”, else, print both the results and “EX-OR logic expression is not verified”.

Objectives:

- Learn how to implement bitwise operations in assembly language along with the C program.

Components required:

Sl No	Description	Specification	Quantity
1 2 3	Raspberry Pi Pico board	RP2040	1
	Micro USB cable	1mtr	1
	Arduino IDE	Software	-

Note:

This needs to be built with the board package of **MbedOS**.

Board and port selection process:

- Click on the option shown “**Select other board and port ...**”.
- In the search window type “**Raspberry**”.
- Then select the Option appearing below
“**Raspberry Pi Pico - Arduino MbedOS RP2040 boards**”.
- Select a **USB port** and upload the program.

If you are able to build it using the “**Raspberry Pi Pico-Raspberry Pi Pico/RP2040**”, continue and you are free to upload it on to the board.

If you try to upload this program with a different Pico board choices than what is already running on the board, the board will become unresponsive.

Remember that when you change this package, from what is running on the board, you will have Upload error and you need to follow the below steps to change the package from one to the other:

- Remove the board from the USB port
- Keep pressing the BOOTSEL button while connecting the board to the USB port of your laptop
- In the Arduino IDE, click the **Tools** menu on the top menu bar, select Board and port (“**UF2 Board**”). Upload the program.
- Upload the code even if the COM port is not recognized by the Arduino IDE
- In case of upload failure, now disconnect the board from the USB port and connect it back
- Now, Arduino IDE should recognize the board and COM port number should appear and try uploading the software now, it should go through successfully.

Experiment 7.1:

Verify the De-morgans theorem with two inputs. If the result is TRUE, print both the results and “De-morgans theorem is verified”, else, print both the results and “De-morgans theorem is not verified”.

```
#include <stdio.h>
#include
<stdlib.h> extern
"C"
{
```

```
unsigned int demorg_1_lhs(unsigned int, unsigned int);  
unsigned int demorg_1_rhs(unsigned int, unsigned int);
```

```

    unsigned int demorg_2_lhs(unsigned int, unsigned int);
    unsigned int demorg_2_rhs(unsigned int, unsigned int);
}

```

```
void setup()

```

```

{
    Serial.begin(9600);
    pinMode(LED_BUILTIN,OUTPUT);
}

```

```
void loop()

```

```

{
    unsigned int val_a = 0xFFFFFFFF;
    unsigned int val_b = 0x00000000;
    unsigned int result1,result2,result3,result4;
    result1 = demorg_1_lhs(val_a,val_b);
    result2 = demorg_1_rhs(val_a,val_b);
    result3= demorg_2_lhs (val_a,val_b);
    result4= demorg_2_rhs (val_a,val_b);
    if (result1 == result2)
    {
        digitalWrite(LED_BUILTIN,HIGH);
        Serial.println("Demorgans theorem1 is verified");
    }
    else
    {
        digitalWrite(LED_BUILTIN,LOW);
        Serial.println("Demorgans theorem1 is not verified");
    }
    if (result3== result4)
    {
        digitalWrite(LED_BUILTIN,HIGH);
        Serial.println("Demorgans theorem2 is verified");
    }
    else
    {
        digitalWrite(LED_BUILTIN,LOW);
        Serial.println("Demorgans theorem2 is not verified");
    }
}

```

```

.global demorg_1_lhs
.global demorg_1_rhs
.global demorg_2_lhs
.global demorg_2_rhs

```

```
.text

```

```

demorg_1_lhs:
    mvn r0,r0 mvn
    r1,r1    and
    r0,r0,r1 bx lr

```

```

demorg_1_rhs :
    orr r0,r0,r1
    mvn r0,r0
    bx lr

```

```

demorg_2_lhs:
mvn r0,r0
mvn r1,r1
ORR r0,r0,r1
bx lr

```

```

demorg_2_rhs:
and r0,r0,r1
mvnr0,r0
bxlr
.end

```

Experiment 7.2:

Perform EX-OR operation with two inputs. If the result is TRUE, print both the results and “EX-OR logic expression is verified”, else, print both the results and “EX-OR logic expression is not verified”.

```

#include <stdio.h>
#include
<stdlib.h>
extern "C" {
    unsigned int xor_asm(unsigned int, unsigned int); // ASM function: ( $\sim A \& B$ ) | ( $A \& \sim B$ )
}

void setup()
{
    Serial.begin(9600);
    pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
    unsigned int val_a = 0xFFFFFFFF;
    unsigned int val_b = 0x00000000;
    // C evaluation:  $A \wedge B$ 
    unsigned int cpp_result = val_a ^ val_b;
    // ASM evaluation: ( $\sim A \& B$ ) | ( $A \& \sim B$ )
    unsigned int asm_result = xor_asm(val_a, val_b);
    if(cpp_result == asm_result) {

        digitalWrite(LED_BUILTIN, HIGH);
        Serial.println("EX-OR Identity is verified:  $A \wedge B = A'B + AB'$ ");
    } else {
        digitalWrite(LED_BUILTIN, LOW);
        Serial.println("EX-OR Identity is NOT verified.");
    }

    delay(2000);
}

```

Asm Program:

```

.global xor_asm
.text
// Inputs: r0 = A, r1 = B
// Computes: ( $\sim A \& B$ ) | ( $A \& \sim B$ )
xor_asm:

```

```
mvn r2, r0    //r2 = ~A
and r2, r2,r1  //r2 = ~A & B

mvn r3, r1    //r3 = ~B
and r3, r3,r0  //r3 = A & ~B
orr r0, r2,r3  //r0 = (~A & B) | (A & ~B)
```

```
.bxlr
end
```

Activity:

Visualize the verification of De Morgan's theorem using LEDs to represent the boolean values.

Experiment 8

1. Write ARM assembly functions to take two unsigned parameters and perform an ADD operation.
2. Write ARM assembly functions to Read the flag bits from the PSR. Return both outputs to the main function.

Objectives:

Learn how to implement compare and conditional branch instructions in assembly language along with the C program.

Components required:

Sl No	Description	Specification	Quantity
1 2 3	Raspberry Pi Pico board	RP2040	1
	Micro USB cable	1mtr	1
	Arduino IDE	Software	-

Table 1: Condition Code Syntax for RP2040

Condition Code	Interpretation	Status Flag State
EQ	Equal / equals zero	Z set
NE	Not equal	Z clear
CS / HS	Carry set / unsigned higher or same	C set
CC / LO	Carry clear / unsigned lower	C clear
MI	Minus / negative	N set
PL	Plus / positive or zero	N clear
VS	Overflow	V set
VC	No overflow	V clear
HI	Unsigned higher	C set and Z clear
LS	Unsigned lower or same	C clear or Z set
GE	Signed greater than or equal	N equals V
LT	Signed less than	N is not equal to V
GT	Signed greater than	Z clear and N equals V
LE	Signed less than or equal	Z set or N is not equal to V
AL	Always (optional)	Any

Note 1:

This needs to be built with the board package of *MbedOS*.

Board and port selection process:

- Click on the option shown “*Select other board and port ...*”.
- In the search window type “*Raspberry*”.
- Then select the Option appearing below.
“*Raspberry Pi Pico - Arduino MbedOS RP2040 boards*”.
- Select a *USB port* and upload the program.

If you are able to build it using the “*Raspberry Pi Pico-Raspberry Pi Pico/RP2040*”, continue and you are free to upload it on to the board.

If you try to upload this program with a different Pico board choices than what is already running on the board, the board will become unresponsive.

Remember that when you change this package, from what is running on the board, you will have Upload error and you need to follow the below steps to change the package from one to the other:

- Remove the board from the USB port.
- Keep pressing the BOOTSEL button while connecting the board to the USB port of your laptop.
- In the Arduino IDE, click the *Tools* menu on the top menu bar, select Board and port (“*UF2 Board*”).
- Upload the program.
- Upload the code even if the COM port is not recognized by the Arduino IDE

- In case of upload failure, now disconnect the board from the USB port and connect it back.
- Now, Arduino IDE should recognize the board and COM port number should appear and try uploading the software now, it should go through successfully.

Note 2: Use Table 1 for the activities.

Experiment 8:

Write ARM assembly functions to take two unsigned parameters and perform an ADD operation and Write ARM assembly functions to Read the flag bits from the PSR. Return both outputs to the main function.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void myPrint(char* fnName, unsigned int result);
```

```
//Define external assembly
```

```
function extern "C" {
```

```
    unsigned int lab8_act_1_1_add(int, int);
```

```
    unsigned int lab8_act_1_1_flag(int,
```

```
    int); bool lab8_act_1_2(int, int);
```

```
    bool lab8_act_1_3(unsigned int, unsigned int);
```

```
    bool lab8_act_1_4(int, int);
```

```
}
```

```
//the setup function runs once when you press reset or power the board
```

```
void setup() {
```

```
    //initialize digital pin LED_BUILTIN as an output.
```

```
    Serial.begin(9600);
```

```
    pinMode(LED_BUILTIN, OUTPUT);
```

```
}
```

```
//the loop function runs over and over again forever
```

```
void loop() {
```

```
    unsigned int add_ret_val;
```

```
    unsigned int flags_ret_val;
```

```
    bool result;
```

```
    unsigned uint_a = 0xFFFFFFFF;
```

```
    unsigned uint_b = 0xFFFFFFFFE;
```

```
    int sint_a = -5;
```

```
    int sint_b = -6;
```

```
    add_ret_val = lab8_act_1_1_add(4, 5);
```

```
    myPrint("lab5_act_1_1_add()", add_ret_val);
```

```
flags_ret_val = lab8_act_1_1_flag(4, 5);
myPrint("lab5_act_1_1_flags()", flags_ret_val);
```

```
if(result = lab8_act_1_2(100, 101)){
    Serial.println("The values are equal.");
    myPrint("lab5_act_1_2()", result);
}
else{
    Serial.println("The values are not equal.");
    myPrint("lab5_act_1_2()", result);
}
```

```
if(result = lab8_act_1_3(uint_a, uint_b)){
    Serial.println("uint_a is higher or same as uint_b.");
    myPrint("lab5_act_1_3()", result);
}
else{
    Serial.println("uint_a is lower than uint_b.");
    myPrint("lab5_act_1_3()", result);
}
```

```
if(result = lab8_act_1_4(sint_a, sint_b)){
    Serial.println("sint_a is lower than sint_b.");
    myPrint("lab5_act_1_4()", result);
}
else{
    Serial.println("sint_a is higher or equal than sint_b.");
    myPrint("lab5_act_1_4()", result);
}
```

```
digitalWrite(LED_BUILTIN, HIGH);
delay(1000);
digitalWrite(LED_BUILTIN, LOW);
delay(1000);
```

```
}//end of loop()
```

```
voidmyPrint(char* fnName, unsigned int printVal){
    Serial.print("The value returned from the assembly fn ");
```

```
Serial.print(fnName);
Serial.print(" : "); Serial.println(printVal);
```

```
Serial.print("The value returned from the assembly fn ");
Serial.print(fnName);
Serial.print(" in Hex: 0x"); Serial.println(printVal, HEX);
Serial.println(); // print an empty line
```

```
}// myPrint(char*, int)
```

```
Asm    progam    .global
lab8_act_1_1_add
.global lab8_act_1_1_flag
.global lab8_act_1_2
.global lab8_act_1_3
.global lab8_act_1_4

.text
@ *****
lab8_act_1_1_add:
add r0, r0, r1 @ R0 <- R0 + R1
bx lr          @ Return from the asm function back to the C prog
```

```
lab8_act_1_1_flag:
add r0, r0, r1 @ R0 = R0 + R1
mrs r0, APSR
mov r0, #0
mvn r1, r0
add r0, r1, #1
mrs r0,
APSR bx lr
```

```
lab8_act_1_2: // Check the two params equal or not
@ This function returns the result after checking
@ if the two parameters passed are equal or not
cmp r0, r1
beq equal
mov r0, #0 @ Return false in R0
b leave_1
equal:
mov r0, #1 @ Return true in R0
leave_1:
bx lr
```

```
lab8_act_1_3: // Check the 1st uint param is
@ higher or same as the 2nd uint param
cmp r0, r1
bhs higher
mov    r0,
#0     b
leave_2
higher:
mov r0, #1
leave_2:
bx lr
```

```
lab8_act_1_4: // Check the 1st signed int param is
@ lower than the 2nd signed int param
```

```
cmp r0, r1
blt less_than
mov r0, #0
b leave_3
less_than:
mov r0, #1
leave_3:
bx lr

.end

@ End of text or code segment
```

Activity:

Write ARM assembly functions to take two unsigned parameters and perform an OR operation.

Experiment 9

Write a program for the Raspberry Pi Pico to transmit and receive data through serial communication (UART) port. Transmitting and Receiving Board (RP2040): UART0, GP0 (UART0 Tx), GP1 (UART0 Rx)

Objectives:

Learn how to transmit and receive the data from one RP2040 board to another RP2040 board through serial communication (UART) port.

Components required:

Sl No	Description	Specification	Quantity
1 2 3	Raspberry Pi Pico board	RP2040	2
4	Micro USB cable	1mtr	1
	Arduino IDE	Software	-
	Jumper wires	M-M	5

Note:

Board and port selection process:

- Click on the option shown “**Select other board and port ...**”.
- In the search window type “**Raspberry**”.
- Then select the Option appearing below.
“**Raspberry Pi Pico-Raspberry Pi Pico/RP2040**”.
- Select a **USB port** and upload the program.

Experiment:

Write a program to transmit and receive the data through serial communication (UART) port as per the following specifications.

Sl No	Raspberry Pi Pico Board	UART	GPIO pin	GP0(UART0
1 2	Tx board (RP2040)	UART0	Tx) &	
	Rx board (RP2040)	UART0	GP1(UART0 Rx) GP0(UART0 Tx) &	
			GP1(UART0 Rx)	

Transmitter Program

```
*PICO_DEFAULT_UART_BAUD_RATE 115200
```

Make sure that Raspberry Pi Pico/RP2040 package is chosen and not the Mbed OS package

HW connections: Connect this Rx board's pin 2 (GPIO 1, RX0) to the Tx board's Pin no. 1 (GPIO 0, TX0), note that UART0 of both the boards are used to communicate the commands from Tx board to Rx board. Tx board receives the command from the user on Serial (USB) port and passes to the Rx board through UART0. The Rx board uses `uart_getc()` SDK function to receive the commands from the UART0 device.

Caution: Though only Tx board is sending data to Rx board, make sure that both Tx and Rx pins are cross connected between Tx and Rx boards. Connect the GND (pin no. 3) of both the boards connected too. It blinks the built-in LED of the RX board connected through UART0. The program runs on Rx board. It receives commands from tx board (USB Serial interface) It receives the following commands (single letter) on UART0 from the TX board The command controls the blinking rate of its built-in LED. Commands: 'o' or 'O': ON, 'f' or 'F': OFF, from 1 to 9 -> blink at 100 to 900 msecs

Program:

```
*/  
  
#include <stdio.h>  
#include "pico/stdlib.h"  
#include "hardware/uart.h"  
#include "hardware/irq.h"  
  
bool LED_ON_flag = false; // Default keep it blinking at default rate 1 sec  
  
static int delayVal = 1000; // Default it is set to 1 second  
  
#define CR_VAL 10  
  
//The command coming in on Serial 0 from the serial monitor of this board  
//to be sent on Serial 1 to the other board to control the built-in LED on the  
//other board int incomingCMD = 0;  
  
int ledCMD = 0; // It has the command from the other board to control this LED  
  
//the setup function runs once when you press reset or power the board  
void setup() {  
  
    Serial.begin(115200); // USB Serial port  
    uart_init(uart0, 115200);  
  
    //Set the GPIO pin mux to the UART0 - 0 is TX0, 1 is RX0  
    gpio_set_function(0, GPIO_FUNC_UART);  
    gpio_set_function(1, GPIO_FUNC_UART);  
  
    //Blink this LED based on the command from the other board  
    pinMode(LED_BUILTIN, OUTPUT);  
    digitalWrite(LED_BUILTIN, HIGH);  
}  
  
//the loop function runs over and over again forever  
void loop() {  
    if(LED_ON_flag)  
        digitalWrite(LED_BUILTIN, HIGH); // keep it always ON without any blinking
```

```

else if (delayVal != 0){
    digitalWrite(LED_BUILTIN, HIGH);
    delay(delayVal);
    digitalWrite(LED_BUILTIN, LOW);
    delay(delayVal);
} else
    digitalWrite(LED_BUILTIN, LOW); // keep it always OFF without any blinking

char rx_char; // received through Serial USB port
// Check if there is any command from the Serial Monitor sent by the user
// Receive it and send it across to the other board through Serial 1
if (Serial.available() > 0) {
    // read the incoming command byte:
    incomingCMD = Serial.read();

    // say what you got:
    Serial.print("Serial: Received the command: ");
    Serial.println(incomingCMD);
    // if it is not a carriage return then send it across to the other board
    // through uart 0
    if (incomingCMD != CR_VAL)
        uart_putc(uart0, incomingCMD); // send it on uart0 to other board
}

} // end of loop()

```

Receiver Program

```

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/uart.h"

#define BUFFER_SIZE 100
char rxBuffer[BUFFER_SIZE];
int rxIndex = 0;

void setup() {
    Serial.begin(115200); // USB Serial for PC communication
    uart_init(uart0, 115200); // UART0 initialization
    gpio_set_function(0, GPIO_FUNC_UART); // TX
    gpio_set_function(1, GPIO_FUNC_UART); // RX

    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, LOW);

```

```

Serial.println("Bi-directional UART ready. Type to send:");
}

void loop() {
    // ... Sending user-typed input over UART
    if (Serial.available()) {
        String inputStr = Serial.readStringUntil('\n');
        inputStr.trim(); // remove newline/whitespace

        for (int i = 0; i < inputStr.length(); i++) {
            uart_putc(uart0, inputStr[i]);
        }
        uart_putc(uart0, '\n'); // End of message
        Serial.print("Sent: ");
        Serial.println(inputStr);
    }

    // ... Receiving UART data and printing to Serial Monitor
    while (uart_is_readable(uart0)) {
        char c = uart_getc(uart0);
        if (c == '\n' || rxIndex >= BUFFER_SIZE - 1) {
            rxBuffer[rxIndex] = '\0'; // End string
            Serial.print("Received: ");
            Serial.println(rxBuffer);

            // Optional: blink LED for "ping"
            if (strcmp(rxBuffer, "ping") == 0) {
                digitalWrite(LED_BUILTIN, HIGH);
                delay(100);
                digitalWrite(LED_BUILTIN, LOW);
            }

            rxIndex = 0;
        } else {
            rxBuffer[rxIndex++] = c;
        }
    }
}

```

Procedure:

- Connect the board-1 pin 1 (GP0, UART0 Tx) to the board-2 pin 2 (GP1, UART0 Rx). Likewise connect the board-1 pin 2 (GP1, UART0 Rx) to the board-2 pin 1 (GP0, UART0 Tx).
- Here, the Transmit (Tx) board will be the board-1 and the Receive (Rx) board will be the board-2.
- Connect the GND (pin 3) pin of the board-1 to GND (pin 3) pin of the board-2.
- Upload the program, activity_1_Tx.ino into the Tx board by connecting it to the system using the USB interface.
- Upload the program, activity_1_Rx.ino into the Rx board by connecting it to the system using the USB interface.
- Caution: If you are connecting both the boards to a same system (laptop) then you need to be careful about which USB COMx port refers to which board (Tx or Rx) and upload the relevant code into them.
- Now, you can give the following commands from the Serial monitor of the Tx board, to control the built-in LED of the Rx board, which is connected to the Tx board through the serial/UART interface.
 - O or o: To make the built-in LED of Rx board always ON
 - F or f: To make the built-in LED of Rx board always OFF
 - 1 to 9: To make the built-in LED of Rx board to blink at the rate from 100 msecs to 900 msecs.

- Any other invalid commands are ignored.
- Note: You need either press enter or control-enter after one of the above commands are given on the Serial monitor.
- The same set of commands must be verified by performing the below activities after making necessary changes to the software of both Tx and Rx boards and also the circuit connecting both boards using three wires and external LEDs as per the activities given below.

Activity:

1. Write a program to transmit and receive the data through serial communication (UART) port as per the following specifications.

Sl No	1	2	Raspberry Pi Pico Board Tx	UART UART0 UART0	G
			board (RP2040)		P
			Rx board (RP2040)		I
					O
					p
					i
					n
					G
					P
					1
					6
					(
					U
					A
					R
					T
					0
					T
					x
					)
					&
					G
					P
					1
					7
					(
					U
					A
					R
					T
					0
					R
					x
					)
					G
					P
					1
					6
					(
					U
					A
					R
					T
					0
					T
					x
					)
					&
					G
					P
					1
					7
					(
					U

$$\begin{matrix} A & T & R &) \\ R & 0 & x & \end{matrix}$$

2. Write a program to transmit and receive the data through serial communication (UART) port as per the following specifications.

SI No	1	2
Raspberry Pi Pico Board Tx board (RP2040)	UART	UART1 UART1
Rx board (RP2040)		

3. Write a program to transmit and receive the data through serial communication (UART) port as per the following specifications.

SI No 1 2	Raspberry Pi Pico Board Tx board (RP2040) Rx board (RP2040)	UART UART0 UART1	G P I O p i n G P 0 (U A R T 0 T x) & G P 1 (U A R T 0 R x) G P 4 (U A R T 1 T x) & G P 5 (U A R T 1 R x)
-----------	---	------------------	---

Experiment 10

10.1 Write a program to control one slave device from one master device using serial communication through the I2C port with the following specifications. Master Slave Board (RP2040): I2C0, GPIO pins: GP0 (I2C0 SDA), GP1 (I2C0 SCL) 10.2 Learn how to connect and control multiple slaves from a single master through I2C serial communication port.

Objectives:

Learn how to connect and control multiple slaves from a single master through I2C serial communication port.

Components required:

Sl No	Description	Specification	Quantity
1 2 3	Raspberry Pi Pico board	RP2040	4
4	Micro USB cable	1mtr	4
	Arduino IDE	Software	-
	Jumper wires	M-M	5

Note:

Board and port selection process:

- Click on the option shown “*Select other board and port ...*”.

- In the search window type “*Raspberry*”.
- Then select the Option appearing below.
“*Raspberry Pi Pico-Raspberry Pi Pico/RP2040*”.

- Select a *USB port* and upload the program.

Experiment 10.1:

Write a program to control the one slave from one master through serial communication (I2C) port as per the following specifications.

Sl No	Raspberry Pi Pico Board	I2C	GPIO pin GP0(I2C0 SDA) &
1 2	Master board (RP2040)	I2C0	GP1(I2C0 SCL) GP0(I2C0 SDA) &
	Slaveboard (RP2040)	I2C0	GP1(I2C0 SCL)

Commands (Slave1):

- O or o: To make the built-in LED of slave board always ON
- F or f: To make the built-in LED of slave board always OFF
- 1 to 9: To make the built-in LED of slave board to blink at the rate from 100 msecs to 900 msecs.
- Any other invalid commands are ignored.

MasterCode (Master board)

This code will run on the Raspberry Pi Pico that acts as the I2C master.

```
#include <Wire.h>
```

```

// I2C address of the slave
#define SLAVE_ADDRESS 0x08

void setup() {
    // Initialize I2C communication as Master
    Wire.begin();
    // Initialize serial communication for debugging purposes
    Serial.begin(9600);
    while (!Serial) {
        ; // wait for serial port to connect. Needed for native USB port only
    }
    Serial.println("Master setup done.");
}

void loop() {
    if (Serial.available()) {
        char command = Serial.read();
        // Send the command to the slave
        Wire.beginTransmission(SLAVE_ADDRESS);
        Wire.write(command);
        Wire.endTransmission();

        // Print to serial monitor
        Serial.print("Sent command: ");
        Serial.println(command);
    }
}

```

Slave Code (Slave board)

This code will run on the Raspberry Pi Pico that acts as the I2C

```

slave. #include <Wire.h>

// I2C address of the slave
#define SLAVE_ADDRESS 0x08
#define LED_PIN 25 // Built-in LED pin

int blinkRate = 0;

void setup() {
    // Initialize the built-in LED pin as an output
    pinMode(LED_PIN, OUTPUT);

    // Initialize I2C communication as Slave
    Wire.begin(SLAVE_ADDRESS);
    // Register a function to be called when data is received
    Wire.onReceive(receiveEvent);
    // Initialize serial communication for debugging purposes
    Serial.begin(9600);
    while (!Serial) {
        ; // wait for serial port to connect. Needed for native USB port only
    }
    Serial.println("Slave setup done.");
}

void loop() {
    if (blinkRate > 0) {
        digitalWrite(LED_PIN, HIGH);
    }
}

```

```

    delay(blinkRate);
    digitalWrite(LED_PIN, LOW);
    delay(blinkRate);
  }
}

//Function to be called when data is received
void receiveEvent(int howMany) {
  while (Wire.available()) {
    char command = Wire.read();
    Serial.print("Received command: ");
    Serial.println(command);
    switch (command) {

      case 'O':
      case 'o':
        digitalWrite(LED_PIN, HIGH);
        blinkRate = 0;
        break;
      case 'F':
      case 'f':
        digitalWrite(LED_PIN, LOW);
        blinkRate = 0;
        break;
      case '1':
      case '2':
      case '3':
      case '4':
      case '5':
      case '6':
      case '7':
      case '8':
      case '9':
        blinkRate = (command - '0') * 100;
        break;
      default:
        // Ignore invalid commands
        break;
    }
  }
}

```

Note1: You need either press enter or control-enter after one of the above commands are given on the Serial monitor.

Procedure:

- Connect the board-1 pin 1 (GP0, I2C0 SDA) to the board-2 pin 1 (GP0, I2C0 SDA). Likewise connect the board-1 pin 2 (GP1, I2C0 SCL) to the board-2 pin 2 (GP1, I2C0 SCL).
 - Here, the master board will be the board-1 and the slave board will be the board-2.
 - Upload the program, *Activity_1_master.ino* into the master board by connecting it to the system using the USB interface.
 - Upload the program, *Activity_1_slave.ino* into the slave board by connecting it to the system using the USB interface.
 - Caution: If you are connecting both the boards to a same system (laptop) then you need to be careful about which USB COMx port refers to which board (master or slave) and upload the relevant code into them.
 - Now, you can give the following commands from the Serial monitor of the master board, to control the built-in LED of the slave board, which is connected to the master board through the serial/I2C interface.
 - The same set of commands must be verified by performing the below activities after making necessary changes to the software of both masters.
-

- and slave boards and the circuit connecting both boards using three wires and external LEDs as per the activities given.

Experiment 10.2:

Write a program to control the three slaves from one master through serial communication (I2C) port as per the following specifications.

Sl No	1	2	3	Raspberr	Pi	Pico	Board	I2C	I2C0	I2C0	I2C1
				Master board	(RP2040)	Slave1					
				board	(RP2040)	Slave2	board				
				(RP2040)							

G
P
I
O
p
i
n
G
P
1
6
(
I
2
C
0
S
D
A
)
&
G
P
1
7
(
I
2
C
0
S
C
L
)
G
P
1
6
(
I
2
C
0
S
D
A
)
&
G
P
1
7
(
I
2
C
0
S
C

L	2	G	1
)	C	P	S
G	1	1	C
P	S	5	L
1	D	(	)
4	A	I	
(	)	2	
I	&	C	

Commands (Slave1):

- ‘o’: To make the built-in LED of slave board always ON
- ‘f’: To make the built-in LED of slave board always OFF.
- ‘1’ to ‘5’: To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.

Commands (Slave2):

- ‘O’: To make the built-in LED of slave board always ON
- ‘F’: To make the built-in LED of slave board always OFF.
- ‘a’ to ‘e’: To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.

Commands (Slave3):

- ‘H’: To make the built-in LED of slave board always ON
- ‘L’: To make the built-in LED of slave board always OFF.
- ‘A’ to ‘E’: To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.

Master Code (Master board) This code runs on the Raspberry Pi Pico that acts as the I2C master. It sends

commands to the three slaves. #include <Wire.h> extern TwoWire Wire1; // I2C addresses of the slaves

#define SLAVE1_ADDRESS 0x08

#define SLAVE2_ADDRESS 0x09

#define SLAVE3_ADDRESS 0x0A

void setup() {

// Initialize I2C communication as Master on I2C0 and I2C1

Wire.begin(); // I2C0 on GP16 (SDA) and GP17 (SCL)

Wire1.begin(); // I2C1 on GP14 (SDA) and GP15 (SCL)

// Initialize serial communication for debugging purposes

Serial.begin(9600);

while (!Serial) {

; // wait for serial port to connect. Needed for native USB port only

}

```

    Serial.println("Master setup done.");
}
void loop() {
    if(Serial.available()) {
        char command = Serial.read();
        if(command == 'o' || command == 'f' || (command >= '1' && command <= '5')) {
            //Send the command to Slave1
            Wire.beginTransmission(SLAVE1_ADDRESS);
            Wire.write(command);
            Wire.endTransmission();
            Serial.print("Sent to Slave1: ");
            Serial.println(command);
        } else if (command == 'O' || command == 'F' || (command >= 'a' && command <= 'e')) {
            //Send the command to Slave2
            Wire1.beginTransmission(SLAVE2_ADDRESS);
            Wire1.write(command);
            Wire1.endTransmission();
            Serial.print("Sent to Slave2: ");
            Serial.println(command);
        } else if (command == 'H' || command == 'L' || (command >= 'A' && command <= 'E')) {
            //Send the command to Slave3
            Wire1.beginTransmission(SLAVE3_ADDRESS);
            Wire1.write(command);
            Wire1.endTransmission();
            Serial.print("Sent to Slave3: ");
            Serial.println(command);
        } else {
            Serial.println("Invalid command");
        }
    }
}
}

```

Slave1 Code

This code runs on the first Raspberry Pi Pico acting as Slave1.

```

#include <Wire.h>

// I2C address of the slave
#define SLAVE_ADDRESS 0x08
#define LED_PIN 25 // Built-in LED pin

int blinkRate = 0;

void setup() {
    // Initialize the built-in LED pin as an output
    pinMode(LED_PIN, OUTPUT);

    // Initialize I2C communication as Slave on I2C0
    Wire.begin(SLAVE_ADDRESS);
    // Register a function to be called when data is received
    Wire.onReceive(receiveEvent);
    // Initialize serial communication for debugging purposes
    Serial.begin(9600);
    while (!Serial) {
        ; // wait for serial port to connect. Needed for native USB port only
    }
}

```

```

    Serial.println("Slave1 setup done.");
}

void loop() {
    if(blinkRate > 0) {
        digitalWrite(LED_PIN, HIGH);
        delay(blinkRate);
        digitalWrite(LED_PIN, LOW);
        delay(blinkRate);
    }
}

//Function to be called when data is received
void receiveEvent(int howMany) {
    while (Wire.available()) {
        char command = Wire.read();
        Serial.print("Received command: ");
        Serial.println(command);
        switch (command) {

            case 'o':
                digitalWrite(LED_PIN, HIGH);
                blinkRate = 0;
                break;
            case 'f':
                digitalWrite(LED_PIN, LOW);
                blinkRate = 0;
                break;
            case '1':
            case '2':
            case '3':
            case '4':
            case '5':
                blinkRate = (command - '0') * 100;
                break;
            default:
                // Ignore invalid commands
                break;
        }
    }
}

```

Slave2 Code

This code runs on the second Raspberry Pi Pico acting as Slave2.

```

#include <Wire.h>
extern TwoWire Wire1;
// I2C address of the slave
#define SLAVE_ADDRESS 0x09
#define LED_PIN 25 // Built-in LED pin
int blinkRate = 0;

void setup() {

    // Initialize the built-in LED pin as an output
    pinMode(LED_PIN, OUTPUT);

    // Initialize I2C communication as Slave on I2C1

```

```

Wire1.begin(SLAVE_ADDRESS);
//Register a function to be called when data is received
Wire1.onReceive(receiveEvent);
//Initializes serial communication for debugging purposes
Serial.begin(9600);
while (!Serial) {
    ;//wait for serial port to connect. Needed for native USB port only
}
Serial.println("Slave2 setup done.");
}

void loop() {
    if(blinkRate > 0) {
        digitalWrite(LED_PIN,HIGH);
        delay(blinkRate);
        digitalWrite(LED_PIN,LOW);
        delay(blinkRate);
    }
}

//Function to be called when data is received
void receiveEvent(int howMany) {
    while (Wire1.available()) {
        char command=Wire1.read();
        Serial.print("Received command: ");
        Serial.println(command);
        switch (command) {

            case 'O':
                digitalWrite(LED_PIN,HIGH);
                blinkRate = 0;
                break;
            case 'F':
                digitalWrite(LED_PIN,LOW);
                blinkRate = 0;
                break;
            case 'a':
            case 'b':
            case 'c':
            case 'd':
            case 'e':
                blinkRate=(command-'a' + 1) * 100;
                break;
            default :
                //Ignore invalid commands
                break;
        }
    }
}

```

Slave3 Code

This code runs on the third Raspberry Pi Pico acting as Slave3.

```

#include <Wire.h>
extern TwoWire Wire1;
// I2C address of the slave
#define SLAVE_ADDRESS 0x0A

```

```

#defineLED_PIN25//Built-in LED pin

int blinkRate = 0;

void setup() {
  //Initializethebuilt-inLEDpin as an output
  pinMode(LED_PIN,OUTPUT);

  //InitializeI2Ccommunication as Slave on I2C1
  Wire1.begin(SLAVE_ADDRESS);
  //Registerafunctiontobecalled when data is received
  Wire1.onReceive(receiveEvent);
  //Initializeserialcommunication for debugging purposes
  Serial.begin(9600);
  while (!Serial) {
    ;//waitforserialporttoconnect. Needed for native USB port only
  }
  Serial.println("Slave3setupdone.");
}

void loop() {
  if(blinkRate > 0){
    digitalWrite(LED_PIN,HIGH);
    delay(blinkRate);
    digitalWrite(LED_PIN,LOW);
    delay(blinkRate);
  }
}

//Functiontobecalledwhendata is received
voidreceiveEvent(inthowMany) {
  while (Wire1.available()) {
    charcommand=Wire1.read();
    Serial.print("Receivedcommand: ");
    Serial.println(command);
    switch (command) {

      case 'H':
        digitalWrite(LED_PIN,HIGH);
        blinkRate = 0;
        break;
      case 'L':
        digitalWrite(LED_PIN,LOW);
        blinkRate = 0;
        break;
      case 'A':
      case 'B':
      case 'C':
      case 'D':
      case 'E':
        blinkRate=(command-'A' + 1) * 100;
        break;
      default :
        //Ignoreinvalidcommands
        break;
    }
  }
}

```

Activity 1:

Write a program to control the one slave from one master through serial communication (I2C) port as per the following specifications.

<i>Sl No</i>	<i>Raspberry Pi Pico Board</i>	<i>I2C</i>	<i>GPIO pin</i>
1	Master board (RP2040)	I2C0	GP0(I2C0 SDA) & GP1(I2C0 SCL)
2	Slave board (RP2040)	I2C0	GP0(I2C0 SDA) & GP1(I2C0 SCL)

Commands (Slave1):

- O or o: To make the built-in LED of slave board always ON
- F or f: To make the built-in LED of slave board always OFF
- 1 to 9: To make the built-in LED of slave board to blink at the rate from 100 msecs to 900 msecs.
- o Any other invalid commands are ignored.

Note1: You need either press enter or control-enter after one of the above commands are given on the Serial monitor.

Procedure:

- Connect the board-1 pin 1 (GP0, I2C0 SDA) to the board-2 pin 1 (GP0, I2C0 SDA). Likewise connect the board-1 pin 2 (GP1, I2C0 SCL) to the board-2 pin 2 (GP1, I2C0 SCL).
- Here, the master board will be the board-1 and the slave board will be the board-2.
- Upload the program, *Activity_1_master.ino* into the master board by connecting it to the system using the USB interface.
- Upload the program, *Activity_1_slave.ino* into the slave board by connecting it to the system using the USB interface.
- Caution: If you are connecting both the boards to a same system (laptop) then you need to be careful about which USB COMx port refers to which board (master or slave) and upload the relevant code into them.
- Now, you can give the following commands from the Serial monitor of the master board, to control the built-in LED of the slave board, which is connected to the master board through the serial/I2C interface.
- The same set of commands have to be verified by performing the below activities after making necessary changes to the software of both master and slave boards and also the circuit connecting both boards using three wires or external LEDs as per the activities given.

Activity 2:

Write a program to control the one slave from one master through serial communication (I2C) port as per the following specifications.

<i>Sl No</i>	<i>Raspberry Pi Pico Board</i>	<i>I2C</i>	<i>GPIO pin</i>
1	Master board (RP2040)	I2C1	GP2(I2C1 SDA) & GP3(I2C1 SCL)
2	Slave board (RP2040)	I2C1	GP2(I2C1 SDA) & GP3(I2C1 SCL)

Note2: If you are using I2C1 ports, mention below statements at the beginning of the program.

```
#include<Wire.h>
```

```
extern TwoWire Wire1;
```

and then, replace all the *Wire.xxxx()* with *Wire1.xxxx()*.

Commands (Slave1):

- O or o: To make the built-in LED of slave board always ON
- F or f: To make the built-in LED of slave board always OFF
- 1 to 9: To make the built-in LED of slave board to blink at the rate from 100 msecs to 900 msecs.
- Any other invalid commands are ignored.

Activity 3:

Write a program to control the two slaves from one master through serial communication (I2C) port as per the following specifications.

<i>Sl No</i>	<i>Raspberry Pi Pico Board</i>	<i>I2C</i>	<i>GPIO pin</i>
1	Master board (RP2040)	I2C0	GP8(I2C0 SDA) & GP9(I2C0 SCL)
2	Slave1 board (RP2040)	I2C0	GP8(I2C0 SDA) & GP9(I2C0 SCL)
3	Slave2 board (RP2040)	I2C1	GP10(I2C1 SDA) & GP11(I2C1 SCL)

Commands (Slave1):

- 'o': To make the built-in LED of slave board always ON
- 'f': To make the built-in LED of slave board always OFF
- '1' to '5': To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.

Commands (Slave2):

- 'O': To make the built-in LED of slave board always ON
- 'F': To make the built-in LED of slave board always OFF
- 'a' to 'e': To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.

Activity 4:

Write a program to control the three slaves from one master through serial communication (I2C) port as per the following specifications.

<i>Sl No</i>	<i>Raspberry Pi Pico Board</i>	<i>I2C</i>	<i>GPIO pin</i>
1	Master board (RP2040)	I2C0	GP16(I2C0 SDA) & GP17(I2C0 SCL)
2	Slave1 board (RP2040)	I2C0	GP16(I2C0 SDA) & GP17(I2C0 SCL)
3	Slave2 board (RP2040)	I2C1	GP14(I2C1 SDA) & GP15(I2C1 SCL)
4	Slave2 board (RP2040)	I2C1	GP14(I2C1 SDA) & GP15(I2C1 SCL)

Commands (Slave1):

- 'o': To make the built-in LED of slave board always ON
- 'f': To make the built-in LED of slave board always OFF
- '1' to '5': To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.

Commands (Slave2):

- 'O': To make the built-in LED of slave board always ON
- 'F': To make the built-in LED of slave board always OFF
- 'a' to 'e': To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.

Commands (Slave3):

- 'H': To make the built-in LED of slave board always ON
- 'L': To make the built-in LED of slave board always OFF

- 'A' to 'E': To make the built-in LED of slave board to blink at the rate from 100 msecs to 500 msecs.
- Any other invalid commands are ignored.